Code Review — Snake Game Engine Architecture

Branch: `game-with-engine-architecture` **Date:** 2026-07-22 **Scope:** Full `src/` tree (~140 files: `core/`, `engine/`, `snake-game/`, `raylib-facade/`, `physics/`, `renderer-2d/`, `debug/`, `platform/`), with focus on recent commits (`ecf0a27`..`0e1621d`) and the untracked `src/engine/events/input/` directory.

Executive summary

The engine rewrite (entity/component pipeline, DI container, scene manager, state machine) is structurally sound, but **the game-state layer (`MainMenuState` / `GameplayState` / `GameOverState` / `GameplayStateMachine`) is currently dead code that cannot run without crashing**. The most recent commit, `0e1621d` ("comment out menu state causing crash"), papers over this by permanently disabling entry into the state machine rather than fixing the root cause — two independent, unconditional null-pointer dereferences on the only code path that would ever execute this layer:

1. `GameContext::input` (a `unique_ptr<InputSystem>`) is never constructed anywhere, yet `MainMenuState`, `GameplayState`, and `GameOverState` all unconditionally call `gameContext.input->IsActionPressed(...)`.
2. `GameplayState::snake` / `GameplayState::apple` are never assigned — the real `Snake` / `Apple` objects live only in `GameplayScene`, a separate object with its own ownership graph.

This indicates the scene/state-machine split was left mid-refactor: two parallel, non-communicating models of "what state is the game in" exist side by side, and only one of them (`GameplayScene`'s direct update loop) is actually wired up and working. Continuing to patch symptoms in the `IGameState` chain (as `0e1621d` did) will keep producing bugs like these until someone decides which model survives.

Separately, ownership in the newer entity/component pipeline (`aa84fa3`, "sole owner entity manager") is coherent: `EntityManager` owns entities via `vector<unique_ptr<IEntity>>`, and components hold non-owning raw pointers into that store. The `ComponentDispatcher` / `TypedComponentRegistrar` visitor pattern is clean.

The project **builds cleanly** — every issue below is a latent logic/UB bug, not a compile error, except one construct that compiles but is undefined behavior if ever executed (finding #3).

The untracked `src/engine/events/input/` directory is dead weight: both it and its tracked counterpart in `src/core/events/input/` are empty (0-byte) files, unreferenced anywhere in the codebase or `CMakeLists.txt` — most likely a stray byproduct of the `move-refactor.sh` script added in `ecf0a27`.

Findings

#	Severity	Area	Location	Issue
1	🔴 Critical	Game state / DI wiring	`gameplay-state-machine.hpp:22`	`GameContext::input` never constructed — unconditional null deref in every state's `Update()`
2	🔴 Critical	Game state / scene ownership	`gameplay-state.cpp:63-80`	`GameplayState::snake` / `apple` never assigned — real objects live only in `GameplayScene`
3	🟠 High	Dependency injection	`dependency-injector.cpp:47-51`	UB: two unrelated `const char*` passed to `std::string` 's iterator-range constructor
4	🟡 Medium	Spatial / transform	`transform-component-2d.cpp:36-46`	`GetTransform()` returns a reference to a function-local `static` shared across all instances
5	🟡 Medium	Event bus	`event-bus.cpp:26-35`	Iterating listener map while a listener can `Unsubscribe` mid-publish — iterator invalidation
6	🟡 Medium	Entity registrars	`render-component-registrar.cpp:6-18`	`manager` raw pointer dereferenced with no null check; interfaces type it as nullable
7	🟢 Low	Rendering interfaces	`i-render-component.hpp:18`	`IRenderComponent` inherits `IRenderable` privately (missing `public`)
8	🟢 Low	Application startup	`application.cpp:90-99`	`strdup` 'd window title never freed (one-time leak)
9	🟢 Low	Physics / collision	`collider-component-2d.hpp:22-25`	`const` accessor returns non-const reference to internal collider state
10	🟢 Low	Entity manager	`entity-manager.cpp:29-33`	`RemoveEntity` never erases ID from lifecycle-tracking sets — unbounded growth
11	🟢 Low	Repo hygiene	`src/engine/events/input/` (untracked)	Empty, unreferenced duplicate of `src/core/events/input/`
12	🟢 Low	Architecture	`gameplay-scene.hpp:56`	Scene owns a `GameplayStateMachine` that's now permanently inert dead weight

---

Detailed findings

Finding 1

🔴 Critical — `GameContext::input` is never constructed; every state's `Update()` unconditionally dereferences it

- `src/snake-game/game-state/gameplay-state-machine.hpp:22`
- `src/snake-game/game-state/main-menu-state.cpp:31`
- `src/snake-game/game-state/gameplay-state.cpp:55-63` (four call sites)

- `src/snake-game/game-state/game-over-state.cpp:32`

```
// gameplay-state-machine.hpp
struct GameContext {
  int score = {0};
  std::unique_ptr<SnakeGameSettings> settings = {nullptr};
  std::unique_ptr<Engine::Input::InputSystem> input = {nullptr};   // never assigned anywhere
};
```

```
// main-menu-state.cpp
void MainMenuState::Update(float)
{
  if (this->gameContext.input->IsActionPressed(Engine::Input::Action::Confirm)) {
    this->GetNextState();
  }
}
```

Root cause: `GameContext::input` is default-initialized to `nullptr` and nothing in the codebase ever assigns it (no `InputSystem` is constructed and stored there). Commit `0e1621d` disabled the single call site that would have entered this state machine (`ChangeState(make_unique<MainMenuState>(...))` in `GameplayStateMachine` 's constructor), which hides the crash for `MainMenuState` specifically — but `GameplayState` and `GameOverState` have the identical unconditional dereference and remain reachable via `MainMenuState::GetNextState()` / `GameOverState::GetNextState()` . The fix in `0e1621d` only prevents entry into the chain; it doesn't fix the chain.

Suggested fix: `GameContext.input` needs to be constructed with a real `InputSystem` (which itself needs an `IInputBackend&`) and injected wherever `GameplayStateMachine` / `GameContext` is built — most likely in `SnakeApplication::Initialize()` . Until that wiring exists, no state in this chain is safe to enter, regardless of whether `ChangeState` is re-enabled.

Finding 2

🔴 Critical — `GameplayState::snake` / `apple` are always null; the real objects live in a separate object (`GameplayScene`)

- `src/snake-game/game-state/gameplay-state.hpp:34-35`
- `src/snake-game/game-state/gameplay-state.cpp:63-80`
- Compare: `src/snake-game/game-scenes/gameplay-scene.cpp:36,45`

```
// gameplay-state.cpp — Enter()
void GameplayState::Enter() {
  if (!this->snake) { LOG_FATAL(...); assert(this->snake); }
  if (!this->apple) { LOG_FATAL(...); assert(this->apple); }
  snake->SetActive(true);
  apple->SetActive(true);
```

Root cause: `GameplayState` owns `unique_ptr<Snake> snake` / `unique_ptr<Apple> apple` members, but nothing assigns them (`this->snake =` / `this->apple =` do not appear in `gameplay-state.cpp`). The actual live `Snake` / `Apple` are constructed and owned by `GameplayScene` — a completely separate object with its own graph. In a debug build this asserts/aborts on `Enter()` ; in a release build (`NDEBUG` , `assert` compiled out) it falls straight through to `snake->SetActive(true)` on a null pointer — an immediate segfault.

This is not an isolated bug — it's evidence that the scene/state split is a half-finished refactor. `GameplayScene` and `GameplayState` currently duplicate the same responsibility (own and update the snake/apple) with disjoint object graphs that never talk to each other.

Suggested fix: Decide the single owner of `Snake` / `Apple` — most likely `GameplayScene`, since it already builds them with `SnakeParams` / `AppleParams` from real dependencies — and either delete `GameplayState`'s duplicate members and inject references from the scene, or delete the `MainMenuState` / `GameplayState` / `GameOverState` chain entirely if the intended direction is scene-driven state rather than the older state-machine model. See finding #12 — both models currently exist and neither is complete.

Finding 3

🟠 High — Undefined behavior in the DI container's error path: two unrelated `const char*` passed to `std::string`'s iterator-range constructor

`src/engine/dependency-injection/dependency-injector.cpp:47-51`

```
catch (const std::exception& e) {
  throw std::runtime_error(
      std::string("[DependencyInjector] Failed to resolve dependency: ", e.what())
  );
}
```

Root cause: `std::string` has no `(const char*, const char*)` overload for concatenation — this resolves to the templated iterator-range constructor `basic_string(InputIt first, InputIt last)` with `InputIt = const char*`. It treats the string literal's address and `e.what()`'s address as a `[first, last)` range into a single buffer, but they're two unrelated allocations. This compiles cleanly (verified via full build) but is undefined behavior at runtime — it will walk memory between two unrelated addresses in whichever order they happen to sort, producing garbage-length reads and likely a crash. It fires exactly when `IDependencyInjector::Resolve<T>()` throws — i.e. exactly the diagnostic path engineers will need most once findings #1/#2 above are being debugged.

Suggested fix: `std::string("[DependencyInjector] Failed to resolve dependency: ") + e.what()`, or route through the project's existing logging macros for consistency.

Finding 4

🟡 Medium — `TransformComponent2D::GetTransform()` returns a reference to a function-local `static`, shared across every instance

`src/engine/spatial/components/transform-component-2d.cpp:36-46`

```
Core::Spatial::ITransform2D& TransformComponent2D::GetTransform()
{
  static Engine::Spatial::Transform2D tempTransform(
      this->GetPosition(), this->GetRotation(), this->GetScale()
  );
  tempTransform.position = this->position;
  tempTransform.rotation = this->rotation;
  tempTransform.scale = this->scale;
  return tempTransform;
}
```

Root cause: `tempTransform` is a single function-local `static` shared by every `TransformComponent2D` instance in the program. Calling `GetTransform()` on segment A, then on segment B before A's caller has consumed the returned reference, silently overwrites A's data through the same object. It's used today in `Snake::CheckIfShouldGrow()` (`src/snake-game/game-entities/snake.cpp:111-113`), where it happens to be safe only because the constructor chain eagerly copies values out before another `GetTransform()` call can occur — an incidental, not structural, safety property. Any future caller that holds the reference across a second `GetTransform()` call (e.g. two segments growing the same frame) will silently corrupt data.

Suggested fix: Return `Transform2D` by value instead of `ITransform2D&` , or have the caller supply storage. A function-local `static` "temporary" return is an anti-pattern for a class that is neither immutable nor single-instance.

Finding 5

🟡 Medium — `EventBus::PublishImplementation` iterates the listener map while a listener can mutate it via `Unsubscribe`

`src/engine/events/event-bus.cpp:26-35`

```
void EventBus::PublishImplementation(std::type_index eventType, const std::any& event)
{
  auto eventListeners = this->listeners.find(eventType);
  if (eventListeners == this->listeners.end()) return;
  for (auto& [token, listener] : eventListeners->second) {
    listener(event);
  }
}
```

Root cause: if a listener invoked during this loop calls `Unsubscribe` (for itself or another listener on the same event type), it erases from the `unordered_map` being iterated — undefined behavior, since `erase` invalidates iterators to the removed element and the loop's current iterator may become invalid depending on which element is erased. This is a classic event-bus footgun, easily triggered by "handle event once, then unsubscribe" patterns.

Suggested fix: copy the listener list (or tokens) into a local vector before invoking, or defer erasure until after the publish loop completes.

Finding 6

🟡 Medium — Registrar dereferences a possibly-null manager pointer with no null check

`src/engine/entities/registrars/render-component-registrar.cpp:6-18`, `src/engine/entities/registrars/render-component-registrar.hpp:19-22`

```
void RenderComponentRegistrar::OnRegistered(Core::Rendering::Components::IRenderComponent* compo
{
  manager->Register(component);   // no null check
}
```

Root cause: `manager` is a raw `IRenderComponentManager*` with no assertion, and the constructors up the chain (`EntityManager(IRenderComponentManager* renderManager)`, `EntityComponentPipeline(IRenderComponentManager* renderManager)`) also type it as nullable. Today the one call site (`GameplayScene`'s constructor) always passes a valid address, so this doesn't currently fire — but nothing enforces the invariant, and a future caller (test harness, headless scene) passing `nullptr` will crash inside `OnRegistered` / `OnUnregistered` with no diagnostic.

Suggested fix: take `IRenderComponentManager&` throughout to remove the possibility of null, or add explicit `assert(manager)` / early-return-with-log guards at the dispatch boundary.

Finding 7

🟢 Low — `IRenderComponent` privately inherits `IRenderable` (missing `public`)

`src/core/rendering/components/i-render-component.hpp:18`

```
class IRenderComponent : public Core::Components::IComponent, IRenderable {
```

Root cause: the second base has no access specifier; for a `class`, that defaults to `private`. `IRenderable` is clearly meant as a general polymorphic interface (referenced by name in `engine/ui/render-component-ui-manager.hpp`, `renderer-2d/components/render-component-ui-manager.hpp`, `renderer-2d/render-component-2d-manager.hpp`), but no code currently upcasts `IRenderComponent*` to `IRenderable*`, so this is latent rather than active. It will surface as a compile error (or a silently-failing `dynamic_cast`) the moment something tries to treat render components generically as `IRenderable`.

Suggested fix: `public Core::Components::IComponent, public IRenderable`.

Finding 8

🟢 Low — `strdup`'d window title is never freed

`src/engine/application/application.cpp:90-99`, `src/engine/utils/string-utils.hpp:5-12`

Root cause: `Application::Initialize()` builds a C string via `strdup` / `_strdup`, passes it to `window->CreateWindow(...)`, and never frees it. One-time startup allocation, so impact is negligible, but a `static` helper that returns owning raw memory from a header is a footgun for future callers.

Suggested fix: pass a `std::string` /local buffer instead (raylib's `InitWindow` copies the title internally), removing the heap allocation; or wrap the result in `std::unique_ptr<char[], decltype(&free)>` if a C string is genuinely required.

Finding 9

🟢 Low — `const` accessor returns a mutable reference into internal state

`src/physics/collision/components/collider-component-2d.hpp:22-25`

```
[[nodiscard]] Physics::Collision::RectangleCollider2D& GetCollider() const override
{
  return *this->collider;
}
```

Root cause: the method is `const` (promising no mutation) but returns a non-const reference to the owned `RectangleCollider2D`, whose fields (`transform`, `offset`, `size`) are public and mutable. Nothing stops a future caller from mutating collider state through a `const ColliderComponent2D&`.

Suggested fix: provide a `const RectangleCollider2D& GetCollider() const` plus a separate non-const overload, matching the pattern already used elsewhere (e.g. `SnakeSegment::GetColliderComponent()`).

Finding 10

🟢 Low — Lifecycle-tracking sets grow unbounded; `RemoveEntity` never prunes them

`src/engine/entities/entity-manager.cpp:29-33`, `src/engine/entities/entity-manager.hpp:41-42`, `src/engine/entities/entity-activation-pipeline.cpp:9-26`

Root cause: `EntityManager::RemoveEntity` tears down components and erases the entity from `entities`, but never removes its ID from `beginPlayFiredIds` or `activeLastFrame`. IDs are monotonically increasing and never reused (`Entity::GenerateId()`), so this isn't a collision bug, but it is an unbounded memory leak proportional to total entities ever created — relevant for long play sessions with many spawned/despawned segments and apples.

Suggested fix: erase `entity->GetID()` from both sets inside `RemoveEntity`.

Finding 11

🟢 Low / repo hygiene — Untracked `src/engine/events/input/` is a dead, empty duplicate

`src/engine/events/input/key-pressed-event.hpp`, `src/engine/events/input/key-released-event.hpp` (untracked) vs. `src/core/events/input/key-pressed-event.hpp`, `src/core/events/input/key-released-event.hpp` (tracked)

Root cause: all four files are 0 bytes. Neither path is `#include`d anywhere in `src/`, and `CMakeLists.txt` doesn't reference them. Likely a stray artifact of `scripts/move-refactor.sh` (added in `ecf0a27`, "refactor Transform into Spatial, add rename script") being run for an input-event move that was never finished.

Suggested fix: given the existing pattern (`core/` = interfaces, `engine/` / `raylib-facade/` = implementations), key-press/-release events belong in `core/events/input/`. Delete the untracked `engine/events/input/` directory, and either fill in the `core/` stubs with real event payloads or remove them until needed — right now they're inert clutter one commit away from being shipped.

Finding 12

🟢 **Low / architecture — `GameplayScene` still owns a `GameplayStateMachine` that is now permanently inert**

`src/snake-game/game-scenes/gameplay-scene.hpp:56`, `src/snake-game/game-scenes/gameplay-scene.cpp:54,57`

Root cause: `GameplayScene::Update()` calls `this->stateMachine.Update(deltaTime)` every frame. Because of finding #1, `currentState` is always `nullptr` (the constructor's `ChangeState` call is commented out in `0e1621d`), so `StateMachine::Update` (`engine/state/state-machine.cpp:27-33`) early-returns every frame — the state machine is now dead weight sitting inside `GameplayScene`, alongside the scene's own working `Snake` / `Apple` / `entityManager` logic that runs independently. There are effectively **two competing "what state is the game in" mechanisms** live in the codebase right now (the old `IGameState` / `StateMachine` chain, and the newer `SceneManager` / `GameplayScene` direct-update model — see the dangling `// if (stateMachine.IsGameOver()) transition.SwitchTo("mainMenu");` comment at `gameplay-scene.cpp:57`), and neither is finished.

Suggested fix: this is the architectural decision underlying findings #1 and #2 — resolve it once, rather than patching each symptom. If scene-driven transitions are the intended direction, remove `GameplayStateMachine` / `GameContext` / `MainMenuState` / `GameplayState` / `GameOverState` entirely rather than leaving them as unreachable dead code.

Recommended fix order

1. **Decide the fate of the `IGameState` / `StateMachine` chain vs. the `SceneManager` / `GameplayScene` model** (finding #12) — this is the real bug `0e1621d` was working around. Everything else in the state-machine chain is downstream of this decision.
2. If keeping the `IGameState` chain: wire a real `InputSystem` into `GameContext::input` (finding #1) and reconcile `GameplayState`'s `snake` / `apple` ownership with `GameplayScene`'s (finding #2) before re-enabling `ChangeState`. If dropping it: delete the dead code instead of leaving it commented out.
3. Fix the DI container's UB error path (finding #3) — one-line fix, and it's the exact diagnostic path that will be needed once #1/#2 are being debugged.
4. Delete or finish the empty untracked `src/engine/events/input/` stub (finding #11) before it's accidentally committed.
5. Sweep the remaining Medium/Low findings (#4–#10) — none are urgent individually, but #4 (shared static transform) and #5 (event bus iterator invalidation) are the kind of bug that's cheap to fix now and expensive to debug later once more gameplay code lands.

Tooling suggestions

- Enable `-fsanitize=address,undefined` for at least the debug CMake preset — it would have caught the DI container UB (finding #3) and would catch findings #1/#2 immediately on the first exercised path, rather than only in a Debug build's `assert`.

- Consider a `clang-tidy` pass with `bugprone-*` and `cppcoreguidelines-*` checks enabled — several findings here (private inheritance, const-correctness, `std::string` iterator-range misuse) are exactly what those catches are for.